

PROIECT DISCIPLINA POO

Aplicație Hospital Management System

(C++)

Autor: Gutan Daniel

TEMĂ PROIECT

Descrierea temei

Aplicație de gestiune a unui spital

Sistemul permite administrarea principalelor activități desfășurate într-un spital.

Funcționalități disponibile

Gestionare pacienți

- adăugare pacient;
- modificare date pacient;
- afișare listă pacienți;
- asociere diagnostic.

Gestionare personal medical

- adăugare medic;
- adăugare asistent;
- afișare personal medical;
- administrare informații personal.

Gestionare programări

- creare programare;
- afișare programări;
- validare programări;
- asociere pacient–medic.

Gestionare facturi

- emitere factură;
- calcul cost servicii medicale;
- afișare facturi;

- salvare informații financiare.

Funcționalități suplimentare

- persistență date în format JSON;
- logging pentru operații critice;
- teste unitare;
- utilizarea pattern-ului Factory pentru generarea facturilor.

CUPRINS

CAPITOLUL I

Descriere proiect din punct de vedere al
utilizatorului.....4

CAPITOLUL II

Descriere proiect din punct de vedere al programatorului.....9

CAPITOLUL III

Dezvoltari ulterioare.....17

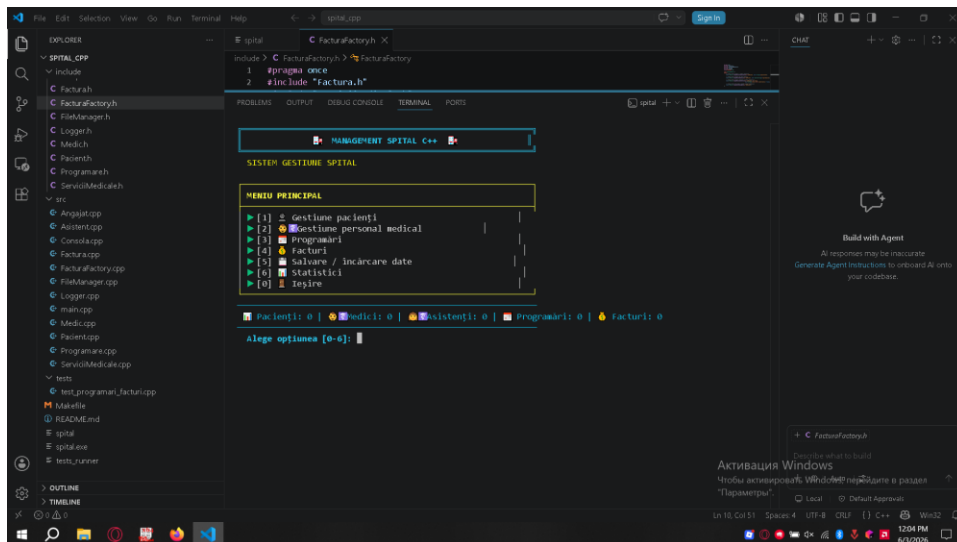
CAPITOLUL I

1. Capitolul I – din punct de vedere al utilizatorului

Pui screenshot-uri din aplicația rulată în consolă:

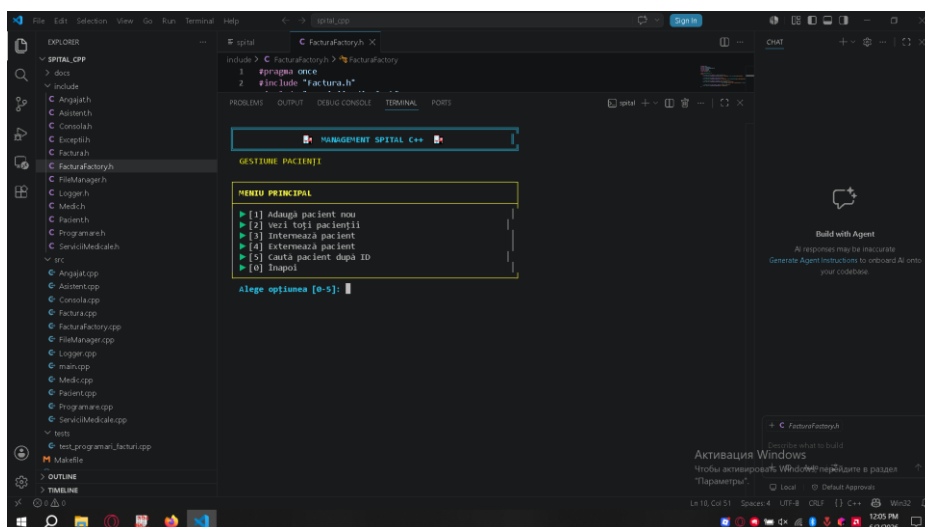
1. Meniul principal

Imaginea prezintă meniul principal al aplicației Hospital Management System. De aici utilizatorul poate accesa principalele module ale sistemului: pacienți, personal medical, programări, facturi, salvarea datelor și statistici. Interfața este realizată în consolă și folosește afișare structurată pentru o navigare mai ușoară.



2. Gestionare pacienți

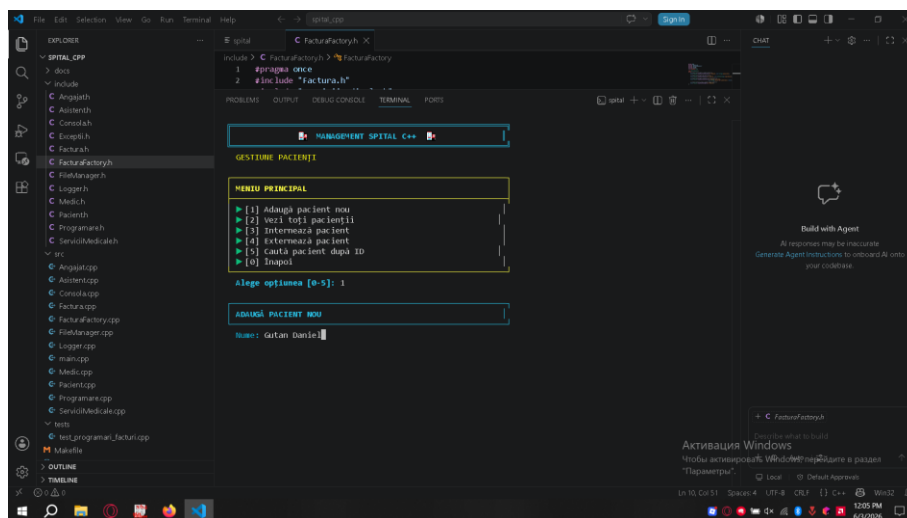
Această secțiune permite administrarea pacienților din cadrul spitalului. Utilizatorul poate adăuga pacienți noi, poate vizualiza lista pacienților existenți și poate modifica informațiile acestora. Datele introduse sunt folosite ulterior în programări, internări și facturi.



3. Adăugare pacient

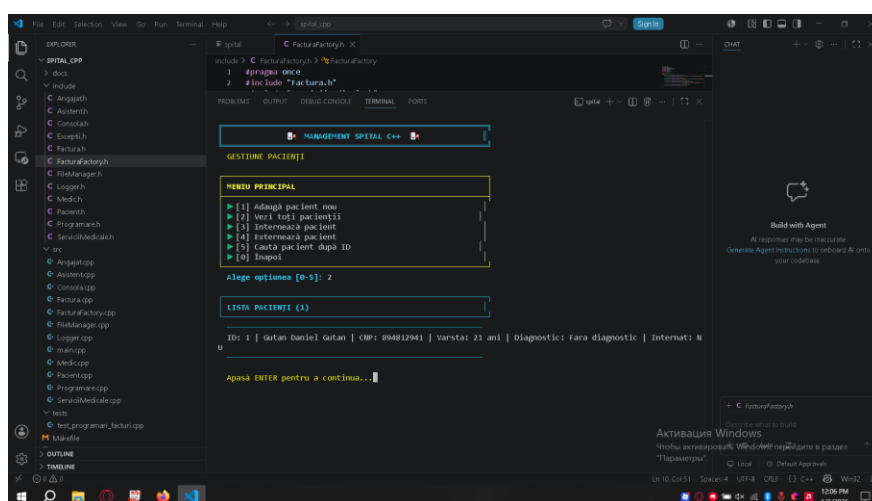
În această imagine este prezentat procesul de introducere a unui pacient nou în sistem. Aplicația solicită informații precum numele,

vârsta și diagnosticul pacientului. Aceste date sunt validate înainte de a fi salvate, pentru a evita introducerea unor valori incorecte.



4. Afisare pacienți

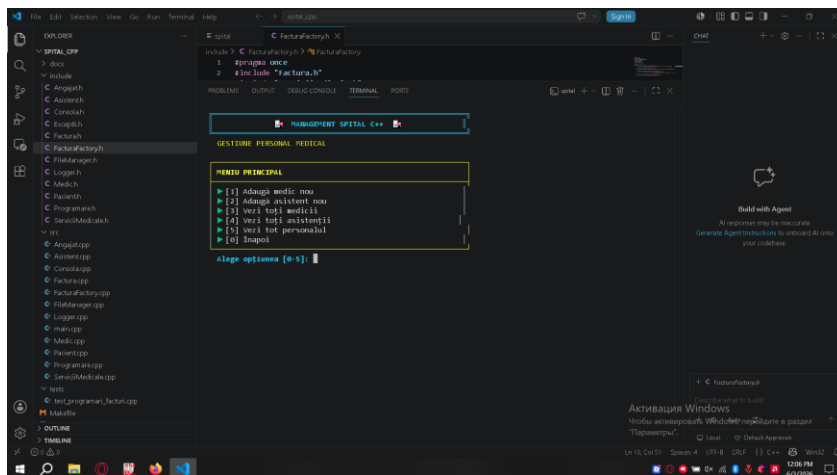
Lista pacienților afișează informațiile salvate în aplicație. Această funcționalitate ajută utilizatorul să consulte rapid pacienții înregistrați și să verifice datele asociate fiecăruia. Informațiile pot fi folosite în continuare pentru programări sau emiterea facturilor.



5. Gestionare personal medical

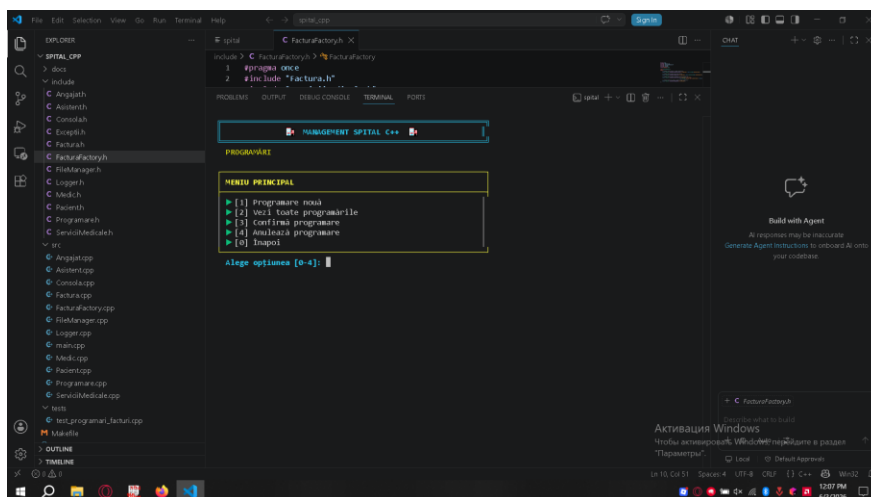
Această secțiune este dedicată personalului medical. Aplicația permite gestionarea medicilor și asistenților, două clase derivate din

clasa de bază Angajat. Astfel, proiectul evidențiază folosirea moștenirii în Programarea Orientată pe Obiecte.

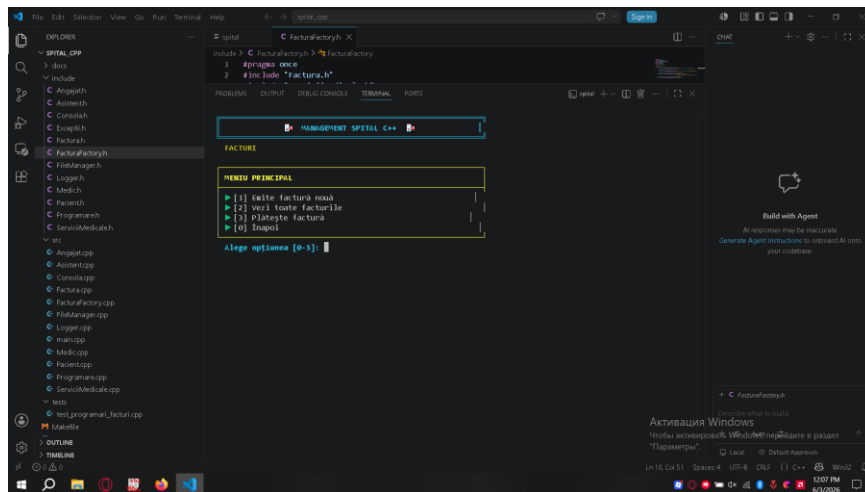


6. Programări

Modulul de programări permite planificarea consultațiilor medicale. O programare leagă un pacient de un medic și conține informații despre dată și oră. În cazul introducerii unor date invalide, aplicația poate trata situația prin excepții personalizate.

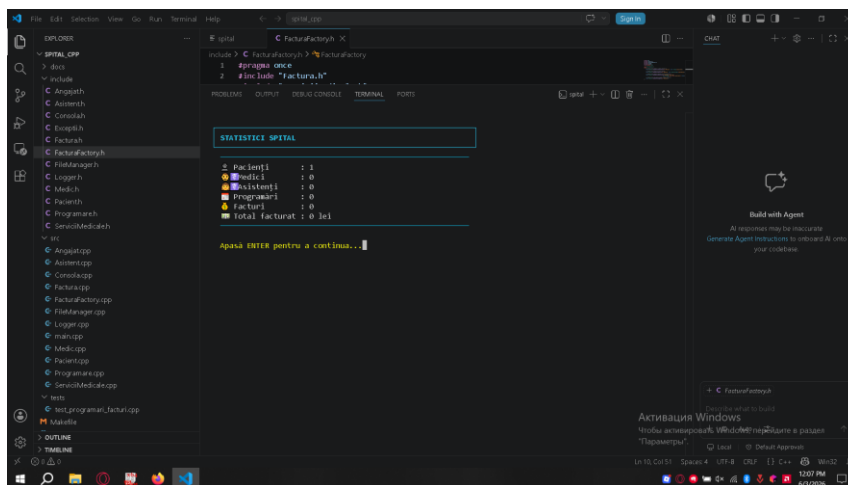


7. Facturi



8. Statistici

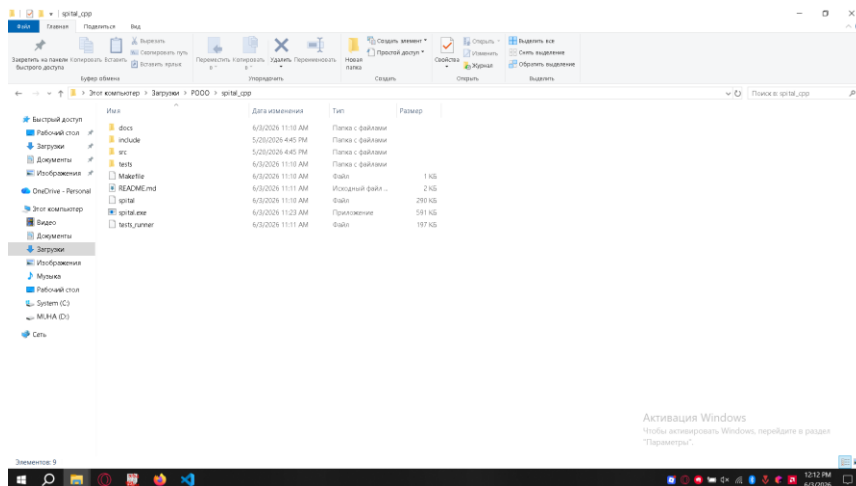
Secțiunea de facturi permite emiterea documentelor de plată pentru serviciile medicale. Costul este calculat pe baza serviciilor selectate, folosind mecanisme de polimorfism. Emiterea facturilor este considerată o operație critică și este înregistrată în sistemul de logging.



CAPITOLUL II

Descriere proiect din punct de vedere al programatorului

- Structura proiectului

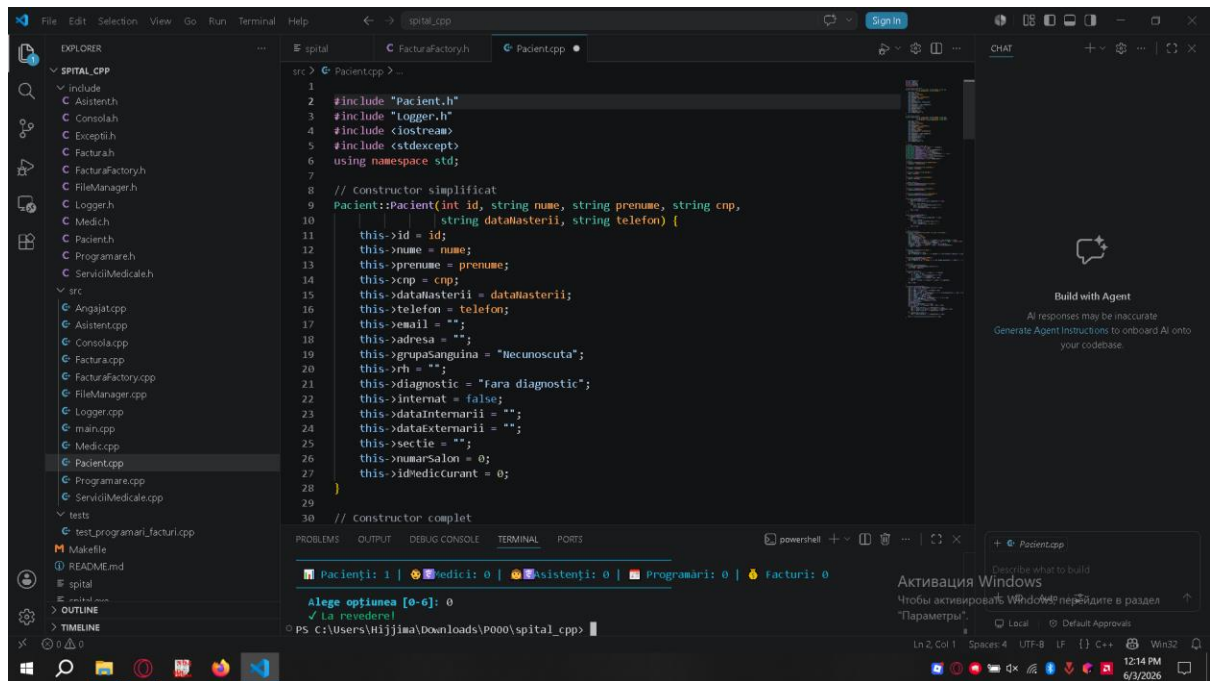


Clasele principale

Screenshot din fișierele:

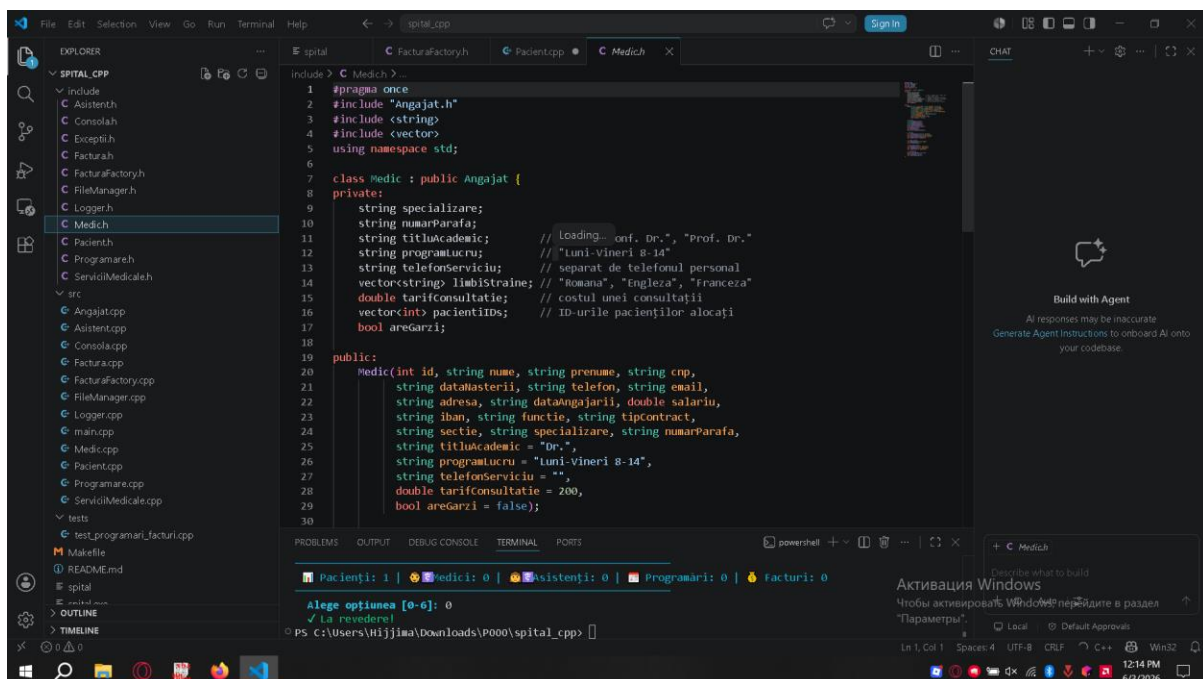
- Pacient.h

Clasa Pacient modelează informațiile asociate unui pacient din cadrul spitalului. Aceasta stochează date precum numele, vârsta și diagnosticul pacientului și oferă metode pentru accesarea și modificarea acestor informații. Clasa reprezintă una dintre entitățile principale ale aplicației și este utilizată în procesul de programare și facturare.



- Medic.h

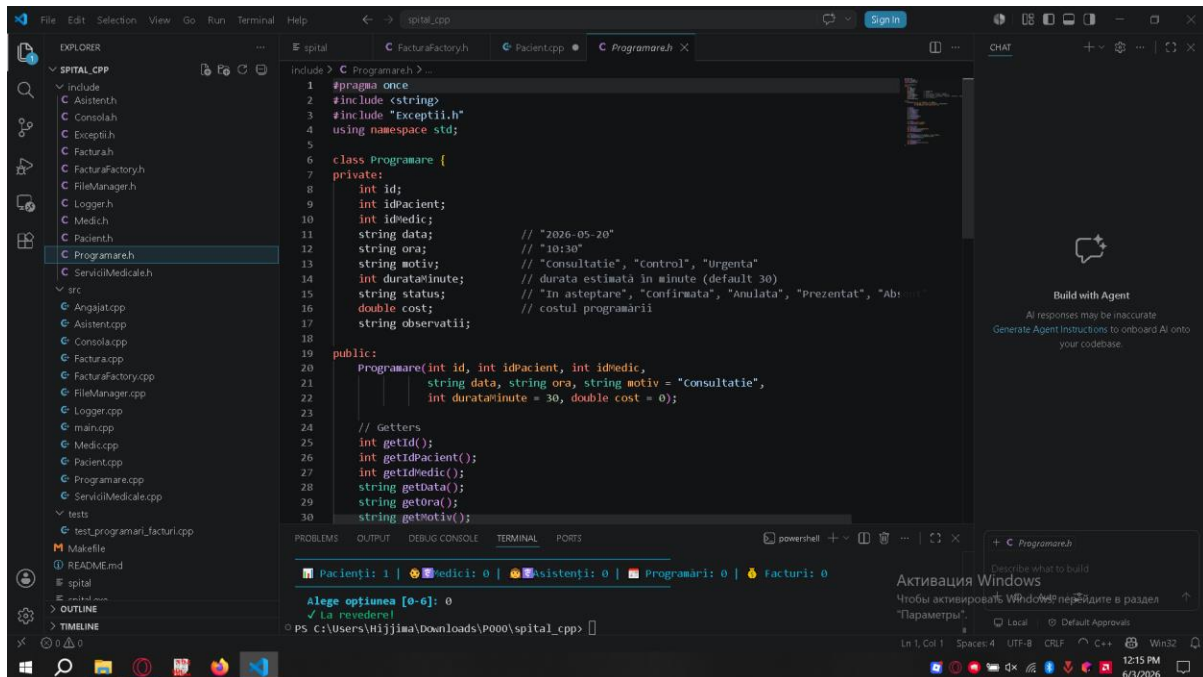
Clasa Medic moștenește clasa Angajat și extinde funcționalitatea acesteia prin adăugarea informațiilor specifice unui medic, cum ar fi specializarea și serviciile medicale oferite. Obiectele acestei clase sunt utilizate în gestionarea programărilor și a consultațiilor.



- Programare.h

Clasa Programare gestionează consultațiile medicale planificate. Ea realizează legătura dintre un pacient și un medic și stochează informații

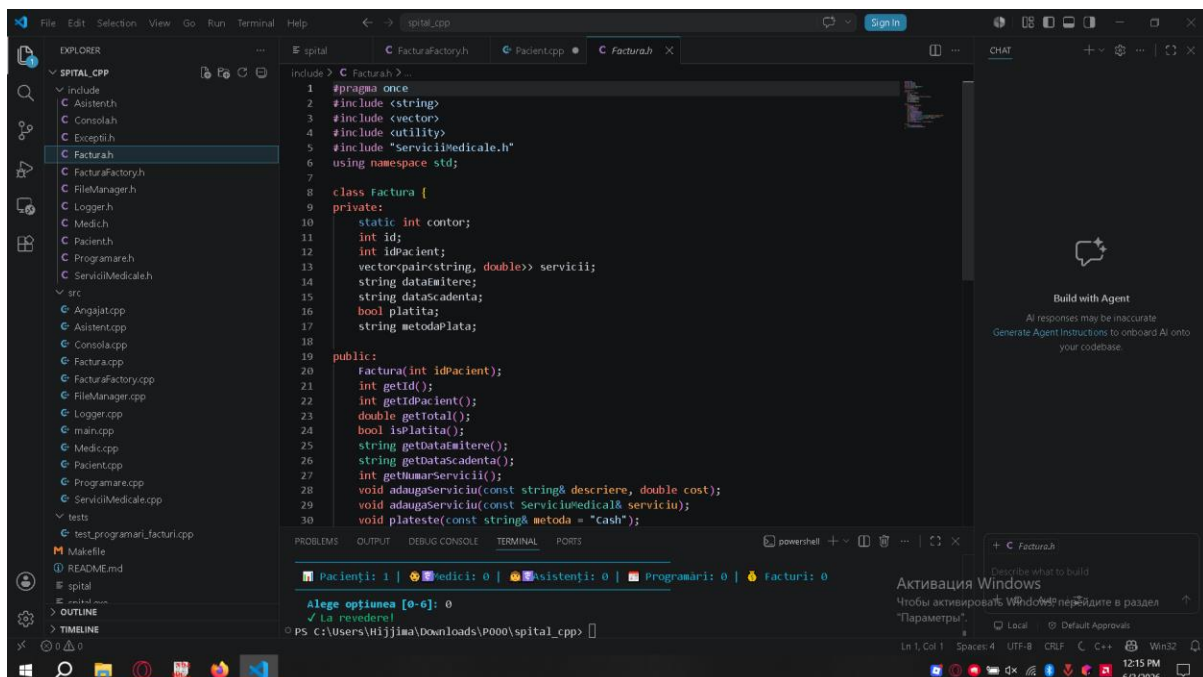
precum data și ora consultației. Pentru validarea datelor introduse sunt utilizate excepții personalizate.



```
1 #pragma once
2 #include <string>
3 #include "Exceptii.h"
4 using namespace std;
5
6 class Programare {
7 private:
8     int id;
9     int idPacient;
10    int idMedic;
11    string data;           // "2026-05-20"
12    string ora;           // "10:30"
13    string motiv;         // "consultatie", "control", "urgenta"
14    int durataIn minute;  // durata estimata in minute (default 30)
15    string status;        // "In asteptare", "confirmata", "Anulata", "Prezentat", "Absent"
16    double cost;          // costul programarii
17    string observatii;
18
19 public:
20    Programare(int id, int idPacient, int idMedic,
21              string data, string ora, string motiv = "consultatie",
22              int durataIn minute = 30, double cost = 0);
23
24    // Getters
25    int getId();
26    int getIdPacient();
27    int getIdMedic();
28    string getData();
29    string getOra();
30    string getMotiv();
```

- **Factura.h**

Clasa Factura este responsabilă pentru calcularea și gestionarea costurilor serviciilor medicale. În cadrul acestei clase este utilizat polimorfismul pentru determinarea costului diferitelor tipuri de servicii. De asemenea, emiterea facturilor este monitorizată prin sistemul de logging implementat în aplicație.

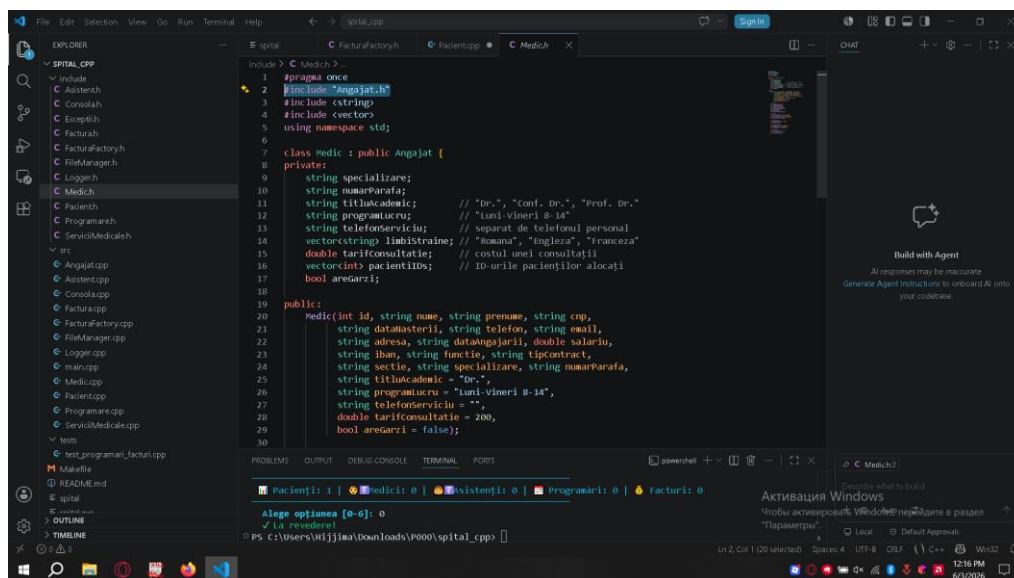


```
1 #pragma once
2 #include <string>
3 #include <vector>
4 #include "utility"
5 #include "ServiciiMedicale.h"
6 using namespace std;
7
8 class Factura {
9 private:
10    static int contor;
11    int id;
12    int idPacient;
13    vector<pair<string, double>> servicii;
14    string dataMitere;
15    string dataScadenta;
16    bool platita;
17    string metodaPlata;
18
19 public:
20    Factura(int idPacient);
21    int getId();
22    int getIdPacient();
23    double getTotal();
24    bool isPlatita();
25    string getDataMitere();
26    string getDataScadenta();
27    int getNumarServicii();
28    void adaugaServiciu(const string& descriere, double cost);
29    void adaugaServiciu(const ServiciiMedical& serviciu);
30    void plateste(const string& metoda = "Cash");
```

- **Moștenirea**

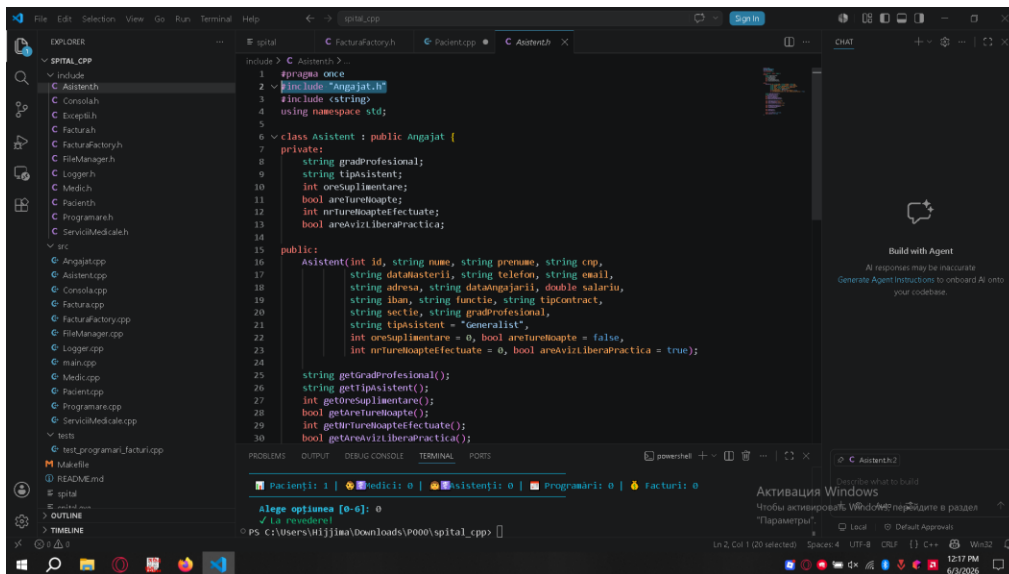
`class Medic : public Angajat`

Clasa Medic moștenește clasa Angajat și extinde funcționalitatea acesteia prin adăugarea informațiilor specifice unui medic, cum ar fi specializarea și serviciile medicale oferite. Obiectele acestei clase sunt utilizate în gestionarea programărilor și a consultațiilor.

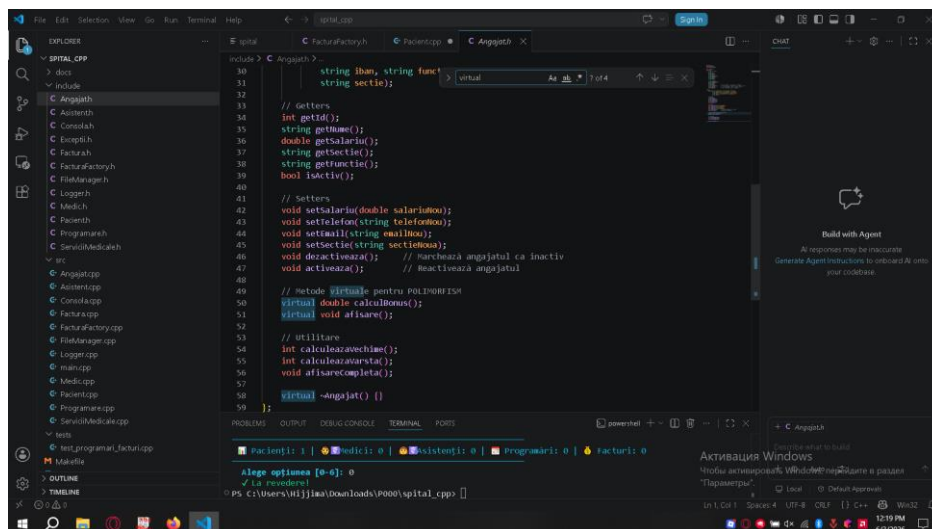


și
`class Asistent : public Angajat.`

Clasa Angajat reprezintă clasa de bază pentru personalul medical. Ea conține informațiile comune tuturor angajaților, precum numele, identificatorul și salariul. Această clasă permite reutilizarea codului prin mecanismul de moștenire.



• Polimorfismul



• Excepții personalizate

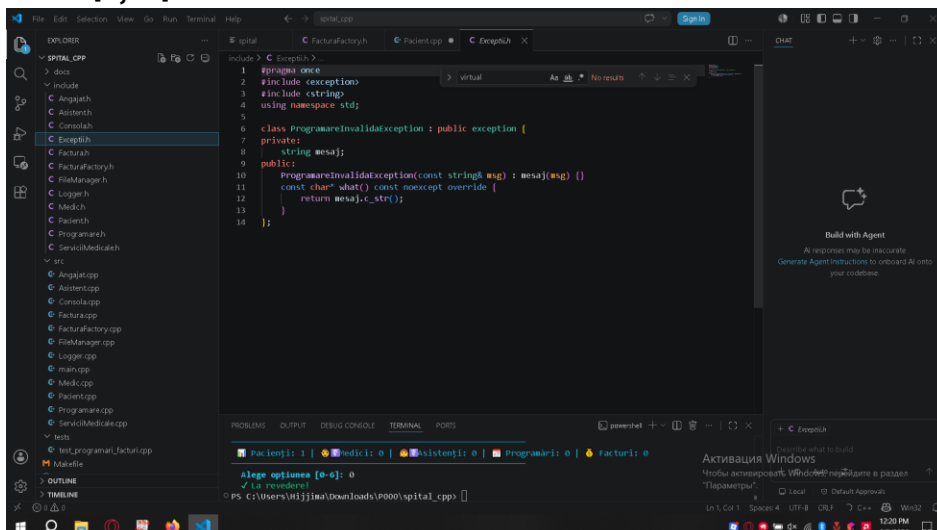
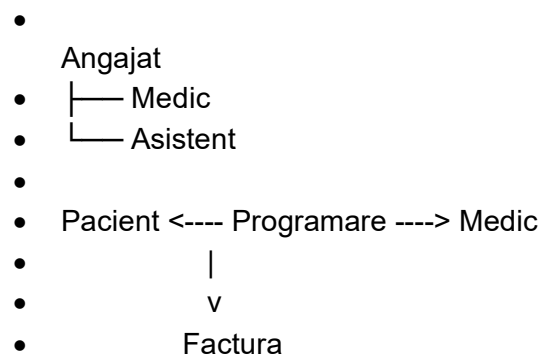


Diagrama UML

Diagrama UML prezintă principalele clase ale aplicației și relațiile existente între acestea. Se observă utilizarea moștenirii prin clasele Medic și Asistent, precum și colaborarea dintre Pacient, Programare și Factura.



CAPITOLUL III

• Dezvoltări ulterioare

• Funcționalități noi care ar fi necesare

- Pentru dezvoltarea aplicației și apropierea acesteia de un sistem utilizat într-un spital real, ar putea fi adăugate următoarele funcționalități:
- Sistem de autentificare pentru utilizatori (administrator, medic, asistent).
- Gestionarea internărilor și externărilor pacienților.
- Evidența saloanelor și a paturilor disponibile.
- Istoric medical complet pentru fiecare pacient.
- Programări într-un calendar interactiv.
- Generarea și exportarea facturilor în format PDF.
- Căutarea și filtrarea avansată a pacienților după diagnostic, vârstă sau medic curant.
- Trimiterea automată a notificărilor pentru programări.
- Gestionarea stocurilor de medicamente și materiale medicale.
- Generarea de rapoarte statistice privind activitatea spitalului.
-

• Modificări de cod sau de tehnologii de implementare

- Pentru îmbunătățirea performanței și a mentenabilității aplicației, ar putea fi realizate următoarele modificări:
- Înlocuirea stocării în fișiere JSON cu o bază de date relațională (MySQL, PostgreSQL sau SQLite).
- Dezvoltarea unei interfețe grafice folosind Qt sau C# WinForms/WPF.
- Implementarea unui sistem Client–Server pentru acces simultan de către mai mulți utilizatori.
- Utilizarea unor pattern-uri suplimentare (Singleton, Observer, Repository).

- Extinderea sistemului de testare prin integrarea framework-ului Google Test.
- Implementarea unui sistem de backup automat al datelor.
- Separarea aplicației pe module independente pentru o întreținere mai ușoară.
- Crearea unei versiuni web accesibile prin browser.
- Implementarea criptării datelor sensibile ale pacienților.
- Integrarea cu sisteme externe de evidență medicală.

BIBLIOGRAFIE

- ✓ Documentația oficială C++
 - o cppreference.com
- ✓ Documentația STL (Standard Template Library)
 - o cplusplus.com
- ✓ Materialele de curs și laborator la disciplina Programare Orientată pe Obiecte
- ✓ Documentația bibliotecii JSON utilizate în proiect (dacă ai folosit nlohmann/json)
 - o JSON for Modern C++
- ✓ Documentația GCC Compiler
 - o GNU GCC Documentation
- ✓ Microsoft Visual Studio Code Documentation
 - o Visual Studio Code Documentation
- ✓ OpenAI ChatGPT – utilizat pentru documentare și clarificarea unor concepte de implementare
 - o ChatGPT
- ✓